

Project summary report

Repository: qc_exciton_LCC
Generated: 2026-05-16
Recipient: jchen9@caltech.edu

This report summarizes, in natural language, what the project's Markdown documentation describes. It is built from README.md, GOALS.md, PLAN.md, AGENT.md, docs/REPO_SUMMARY.md, docs/WORKFLOW.md, and the autonomous-cycle log AUTONOMOUS_LOG.md (cycles 1 through 19, roughly 1,500 lines of running notes from May 16, 2026).

The repository is quantum-chemistry and quantum-resource-estimation tooling for localized-orbital exciton workflows. Its overarching scientific aim, as recorded in GOALS.md, is to build a concrete extension of arXiv:2508.15765 - currently without locality, focused on excitons rather than the full LCC framework. The Coulomb operator is decomposed via tensor hypercontraction (THC) with translational symmetry, and the block-encoding scheme uses direct unitary synthesis on each tensor combined with the space-time trade-offs from arXiv:1812.00954 (Babbush et al.) and the constant-factor improvements from Section III.A of arXiv:2409.11748. The two main scaling regimes targeted are $O(N_k * N^2 / \epsilon)$ Toffolis with $O(\log(N_k * N^2))$ qubits, and $O(\sqrt{N_k} * N^{3/2} / \epsilon)$ Toffolis with $O(\sqrt{N_k * N})$ qubits. An important point is that the effective Hamiltonian considered (especially the BSE case) has norm $O(1)$, much smaller than the bare many-body Hamiltonian whose norm scales with system size.

Repository layout (README.md, docs/REPO_SUMMARY.md, docs/WORKFLO

The active source tree is organized around three areas:

- `src/chem` hosts PySCF-based molecular calculations and localized molecular-orbital (LMO) data generation. The entry point `pyscf_adapter.py` builds a small MoS2 example geometry, runs SCF (RHF, UHF, RKS, UKS), localizes occupied and virtual spaces separately with Boys or Pipek-Mezey, transforms one- and two-electron integrals into the LMO basis, estimates orbital centers, and computes a simple statically screened Coulomb tensor. The one-shot helper `PySCFExcitonDataBuilder` bundles all of this and returns an `LMOData` record. This is described as the most coherent active domain layer.
- `src/integrations` is the home of provider-specific resource-estimation experiments. The active work is concentrated in `src/integrations/qualtran`, which contains: QROAM-backed state preparation with rotation tables (`state_prep_QROAM.py`), block-indexed state preparation conditioned on a block register (`block_state_preparation_QROAM.py`), Householder-reflection unitary synthesis (`unitary_synthesis_QROAM.py`), block-diagonal unitary synthesis (`block_unitary_synthesis_QROAM.py`), the block-interferometer variant (`block_unitary_interferometer_QROAM.py`), shape-only data-loading comparisons across QROM, `SelectSwapQROM`, and `QROAMClean` (`data_loading_comparison.py` and `.ipynb`), and helper utilities for Toffoli, qubit, and ancilla counts in `utils.py`.
- `src/exciton` contains early dataclasses for exciton models, screening providers, benchmark tensors, and a minimal builder. `REPO_SUMMARY.md` explicitly notes this is not yet consistent with the active effective-Hamiltonian and integration work, and should be treated as experimental scaffolding.

The previously active `src/block_encoding`, `src/backends`, `src/algorithms`, `src/oracles`, `src/integrations/pennylane`, and older `qualtran/block_encoding` packages have been removed from the source tree and preserved as tarballs under `archive/`. The test suite is described as transitional: several tests still import deleted modules.

Standard install paths are `python -m pip install -e .` with optional extras `[chem]`, `[qualtran]`, `[dev]`. Helper scripts under `scripts/linux/` (`bootstrap.sh`, `run.sh`) and a WSL wrapper `run_in_wsl.ps1` are provided for chemistry runs.

Scientific goals (GOALS.md)

GOALS.md lists six concrete directions of work:

1. Use quantum tensor train (QTT) compression to further compress data such as the Q indices in THC, or more generally consider decompositions that further save data.
2. Improve constant factors in the quantum algorithms, or scaling where possible.
3. Identify where sparse block-encoding schemes can be applied.
4. Compare density fitting against THC - density fitting has worse scaling but may carry smaller constants or smaller subnormalization.
5. Incorporate non-abelian symmetries.
6. Pursue any further improvements on the final Toffoli complexity, qubit counts, or scaling.

GOALS.md also points to notes/XPRIZE_notes.pdf as a partially error-prone background reference.

PLAN.md describes the autonomous-cycle workflow: inspect repo, choose one small high-value task per cycle, prefer fixing failing tests, lint, coverage, or documentation, implement one focused change, run checks, commit, and update AUTONOMOUS_LOG.md.

AGENT.md governs what an autonomous cycle is allowed to do: edit source, add or update tests, improve docs, refactor small isolated areas, and create small commits. It forbids touching secrets, production infrastructure, IAM, deployment, pushing to main, force pushing, and large deletions.

Cycles 1-5: pinning the synthesis cost structure

AUTONOMOUS_LOG.md records 19 autonomous work cycles on May 16, 2026.

The early cycles focused on locking in the cost structure of the block-unitary synthesis blog so that future constant-factor work cannot silently regress.

Cycle 1 added tests/test_model_resource_counts.py covering the analytic interferometer resource model: ceil_log2, the power-of-two assertion, three hand-checked Toffoli cases for block_unitary_interferometer_toffoli, qubit-count cases, input validation, the Pareto-consistent optimizer, and most importantly a cross-check that the analytic block_unitary_interferometer_count agrees with the Bloq-side estimate_interferometer_resources for matched parameters. 15 tests passed.

Cycle 2 added tests/test_block_unitary_synthesis_equivalence.py (12 tests): the single-block ($n_blocks=1$) limit of BlockUnitarySynthesisQROAM matches the un-blocked UnitarySynthesisQROAM in Toffoli cost at $N=2, 4, 8$, signature shapes are correct, isometry support is exercised, the data-free from_shape path raises DecomposeTypeError, symbolic from_shape retains symbols, non-orthonormal data is rejected, and the adjoint of BlockPrepareHouseholderStateQROAM is an involution.

Cycle 3 added tests/test_block_unitary_synthesis_scaling.py (6 tests). The key invariants pinned: K -linearity ($T_total(K) = K * T_total(K=1)$ exactly), b -affineness, the per-reflection b -slope identity $2*(\log_2(N)+1)$ (independent of n_blocks), shape-only equality to data-bearing constructors, and the default $n_reflections = n_rows$.

Cycle 4 added tests/test_block_unitary_synthesis_amortization.py (5 tests): strict sub-linearity $T(n_blocks) < n_blocks * T(1)$ for $n_blocks \geq 2$ across a (N, b, K) grid; non-increasing $T(n_blocks)/n_blocks$ along doublings; the quadrupling bound $T(4*n_blocks) \leq 2 * T(n_blocks)$; K -independence of the n_blocks ratio; and $1.7 \leq T(64)/T(16) \leq 2.0$ asymptotically - i.e. the QROAMClean $\sqrt{M}$ regime.

Cycle 5 added tests/test_block_unitary_synthesis_b_intercept.py (5 tests) pinning the $b=0$ intercept $I_1(n_blocks, N)$ on the 15-point grid n_blocks in $\{1, 2, 4, 8, 16\}$ times N in $\{4, 8, 16\}$. Once this intercept is tabulated, the full decomposition $T(n_blocks, N, K, b) = K * (2*(\log_2(N)+1)*b + I_1(n_blocks, N))$ is fully characterized except for one lookup.

Cycles 6-11: closed analytic estimator, qubit counts, full grid

Cycle 6 added `SYNTHESIS_PER_REFLECTION_INTERCEPT` and `block_unitary_synthesis_toffoli` to `src/integrations/qualtran/model_resource_counts.py`, returning $K * (2 * (\log_2(N)+1) * b + I_1)$ with `KeyError` off-grid. A cross-check test confirms the analytic estimator equals the Bloq's `QECGatesCost` Toffoli output over the tabulated grid.

Cycle 7 extended the intercept table from 15 to 49 entries - `n_blocks` in `{1,2,4,8,16,32,64}` times `N` in `{4,8,16,32,64,128,256}` - so the analytic estimator covers the parameters the actual `block_unitary_resource_report.py` run uses (default `N=256`, `n_blocks=k^3`).

Cycle 8 added `block_unitary_synthesis_signature_qubits` returning `ceil_log2(n_blocks) + 1 + log2(n_rows) + bitsize` with a Bloq cross-check that confirms it equals `bloq.signature.n_qubits()` exactly across the 49-point grid times three bitsizes. This pins the persistent qubit count, deferring the transient QROAMClean workspace to a later cycle.

Cycle 9 introduced the `SynthesisResourceCount` frozen dataclass and the `block_unitary_synthesis_count` wrapper, composing the Toffoli and signature-qubit helpers into a single record-shaped API that mirrors `block_unitary_interferometer_count`. The cycle also uncovered an upstream `QubitCount` failure in `BlockPRGAViaPhaseGradientQROAM` that prevented tabulating workspace qubits.

Cycle 10 fixed that upstream blocker. Root cause: `BlockPrepareHouseholderStateQROAM.build_composite_bloq` unconditionally called `soqs.pop('block')`, but when `n_blocks=1` the block register has bitsize 0 and is omitted from the signature. The fix pops 'block' with a `None` default and only forwards it when present. A new test file `tests/test_block_unitary_synthesis_qubit_count.py` (5 tests) pins the regression, full-grid `QubitCount` success, the qubit-side single-block equivalence, and workspace monotonicity in both `n_blocks` and `n_rows`.

Cycle 11 added `SYNTHESIS_WORKSPACE_QUBITS`, a 245-point table (49 (`n_blocks`, `n_rows`) cells times 5 bitsizes in `{2,4,8,16,32}`), `block_unitary_synthesis_workspace_qubits` as a lookup, `workspace_qubits` and `total_qubits` fields on `SynthesisResourceCount`, and the load-bearing `test_block_unitary_synthesis_count_total_qubits_matches_bloq` cross-check that the analytic `total_qubits` equals `QubitCount(bloq)` exactly across the full tabulated grid.

Cycles 12-15: report plumbing, consistency guards, power-law fits

Cycle 12 plumbed `block_unitary_synthesis_count` into the `_plot_report` PDF generator. New `SYNTHESIS_PANEL_N_BLOCKS = (1, 2, 4, 8, 16, 32, 64)` and a `_synthesis_panel_records` helper sweep the analytic estimator; `plot_synthesis('toffoli')` and `plot_synthesis('qubits')` were added as new plot pages, plus a `synthesis-table` page; the summary page was updated to describe the new panel. Three new tests including an end-to-end PDF smoke test were added.

Cycle 13 closed an auto-generation gap from cycle 7: the production lookup `SYNTHESIS_PER_REFLECTION_INTERCEPT` and the test-side `REFERENCE` table are now guaranteed equal via `test_reference_table_matches_module_table`, so future grid extensions cannot accidentally update one without the other.

Cycle 14 added `test_block_unitary_synthesis_workspace_table_monotone`, a fast pure-dict structural check that workspace qubits are non-decreasing in `n_blocks` across every `(n_rows, bitsize)` slice (35 slices) and non-decreasing in `n_rows` across every `(n_blocks, bitsize)` slice (35 slices). The docstring explicitly notes that monotonicity in bitsize is not an invariant - 16 of 245 entries show non-monotone bitsize trends driven by the QROAMClean optimizer's discrete block-size choices.

Cycle 15 added a report-visible power-law fit for the synthesis panel: $c * n_blocks^\alpha$ overlaid on both the Toffoli and total-qubit plots, with α printed in the legend and in the text summary page. The pinned values at the canonical report parameters ($N=256$, $b=32$, $K=block_dim=256$) are
 α_t approximately 0.275
 α_q approximately 0.024
both well below 1, confirming QROAMClean amortization is visible in the report. `test_synthesis_panel_power_law_sublinear` asserts both alphas lie in $[0, 1)$.

Cycles 16-19: per-cycle PDF reports and table regeneration

AUTONOMOUS_INBOX.md added a user instruction on 2026-05-16: 'Send me a pdf report each time, with math equations rendered as latex.'
Cycles 16 through 19 built up the reporting infrastructure to satisfy this.

Cycle 16 produced the first per-cycle deliverable (docs/cycle_reports/cycle16_report.pdf, rendered with matplotlib mathtext) and addressed the long-pending cycle-12 recommendation in its intermediate form by adding scripts/regenerate_synthesis_tables.py. The script exposes `extract_intercept` (subtracting the $2 * (\log_2(N) + 1) * b_{\text{ref}}$ slope from `QECGatesCost(bloq).toffoli`) and `extract_workspace` (`QubitCount(bloq) - signature.n_qubits()`), and provides `--check` (default) which diffs regenerated dicts against shipped tables, `--print` which emits paste-ready Python source, and `--no-workspace` to skip the slower sweep. The full run reports 'SYNTHESIS PER REFLECTION INTERCEPT: OK (49 entries match); SYNTHESIS WORKSPACE QUBITS: OK (245 entries match)'. A note in the cycle log records that an attempt to email the report via the existing SMTP path was denied by the harness' auto-mode classifier.

Cycle 17 added a reusable `scripts/autonomous_cycle_report.py` PDF generator that emits a compact three-page cycle report with argument-driven task/changes/checks sections and rendered LaTeX-style equations for $T(M, N, K, b)$, $Q(M, N, b)$, and the sub-linear amortization claim. A smoke test (`tests/test_autonomous_cycle_report.py`) verifies the helper writes a non-empty PDF. Cycle 17 also notes a Git failure ('fatal: Unable to create .git/index.lock: Read-only file system') that prevented committing during that cycle.

Cycle 18 extended the report generator with `DEFAULT_EQUATIONS` and a repeatable `--equation` CLI argument so cycles can render task-specific math rather than always using the default synthesis formulas.

Cycle 19 added a `--summary` mode to `scripts/regenerate_synthesis_tables.py`: log-log least-squares fits $y \sim c * x^{\alpha}$ (no new dependencies), and a formatted summary of canonical-slice fits for $I_1(n_{\text{blocks}}, N=256)$ and $W(n_{\text{blocks}}, N=256, b=32)$. New tests recover $\alpha=2$, $c=3$ from a synthetic quadratic and confirm the fit lines render.

Cycle 20 (PDF present at docs/cycle_reports/cycle20_report.pdf, no formal log entry yet) continued the per-cycle PDF cadence.

Overall status and open work

What is closed:

- The Toffoli cost of BlockUnitarySynthesisQROAM is fully characterized as
$$T(n_blocks, N, K, b) = K * (2 * (\log_2(N) + 1) * b + I_1(n_blocks, N))$$
with I_1 tabulated over a 49-point grid and cross-checked against the Bloq's QECGatesCost.
- The total qubit count is fully characterized as
$$Q = \text{signature_qubits} + \text{workspace_qubits}$$
with signature_qubits given in closed form and workspace_qubits tabulated over a 245-point grid (49 (n_blocks, N) cells times 5 bitsizes), cross-checked against the Bloq's QubitCount.
- K-linearity, b-affineness, the b-slope identity $2 * (\log_2(N) + 1)$, n_blocks/n_rows monotonicity of the workspace table, and sub-linear n_blocks amortization (toward the QROAMClean $\sqrt{M}$ regime) are all pinned as explicit tests.
- The analytic estimator is plumbed into the report PDF generator, with a power-law fit overlay (α_t about 0.275, α_q about 0.024 at $N=256$, $b=32$, $K=256$) and a summary text page.
- Tabulated dicts are regenerable from the Bloq in one command via `scripts/regenerate_synthesis_tables.py`, with a `--summary` mode for empirical scaling diagnostics.
- The upstream QubitCount failure at $n_blocks=1$ in `BlockPrepareHouseholderStateQROAM` has been fixed.
- Per-cycle PDF reports with LaTeX-rendered equations are now standard, supported by a reusable `scripts/autonomous_cycle_report.py` helper.

What remains open (the standing 'next recommended task' since cycle 12):

- Derive $I_1(n_blocks, N)$ and `SYNTHESIS_WORKSPACE_QUBITS` in closed form from QROAMClean's optimizer expression (table length $M = n_blocks * N$, output bitsize b , optimal block size k^* of order $\sqrt{M*b}$, plus per-reflection reflection-about-zero / Hadamard overhead). The regression script provides the oracle; the empirical $I_1 \sim c * \sqrt{M*N}$ and α_T about 0.275 fits at $N=256$ are the asymptotic targets the closed form must reproduce. Once derived, the tabulated dicts become regression caches rather than sources of truth, and the estimator extends to arbitrary parameters without re-running the Bloq.

Beyond this immediate item, GOALS.md still lists QTT compression, sparse block encodings, density-fitting vs THC comparisons, and non-abelian symmetries as larger directions of work. The autonomous cycles to date have focused entirely on goal 2 ('improve constant factors') and goal 6 ('any further improvements on Toffoli/qubit/scaling'), within the block-unitary synthesis bloq.

Key formulas (rendered)

Toffoli cost of BlockUnitarySynthesisQROAM (cycles 5 to 7):

$$T(M, N, K, b) = K [2(\log_2 N + 1) b + I_1(M, N)]$$

Persistent qubit count (cycle 8):

$$Q_{\text{sig}}(M, N, b) = \lceil \log_2 M \rceil + 1 + \log_2 N + b$$

Total qubit count (cycle 11):

$$Q_{\text{tot}}(M, N, b) = Q_{\text{sig}}(M, N, b) + W(M, N, b)$$

Sub-linear amortization (cycle 4):

$$T(M) < M T(1) \quad \text{and} \quad T(4M) \leq 2 T(M)$$

Empirical power-law fit (cycle 15, N=256, b=32, K=256):

$$T \sim c M^{\alpha_t}, \quad \alpha_t \approx 0.275; \quad Q_{\text{tot}} \sim c M^{\alpha_q}, \quad \alpha_q \approx 0.024$$

Targeted scaling regimes (GOALS.md):

$$\mathcal{O}(N_k N^2 / \varepsilon) \text{ Toffoli, } \mathcal{O}(\log(N_k N^2)) \text{ qubits; or } \mathcal{O}(\sqrt{N_k} N^{3/2} / \varepsilon) \text{ Toffoli, } \mathcal{O}(\sqrt{N_k N}) \text{ qubits}$$